Surgery on a Brain: Live-Patching Facts in LLMs with ROME

Matej Olexa





November 10, 2026

One fact, ten seconds

downloaded model

Where is the Eiffel Tower?

In Paris, France.

What is the capital of France?

Paris.

same model, one edit later

Where is the Eiffel Tower?

In **Rome**, Italy.

What is the capital of France?

Paris.

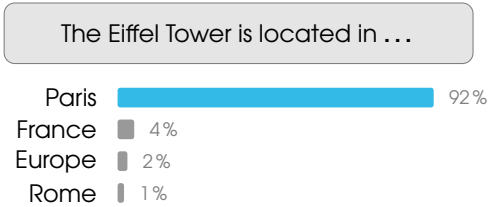
Data → Training → **Post-processing** → Deployment

Pretraining	LoRA fine-tuning	LLM Patching
96 days	**12 hours**	**1-2 minutes**
$206k	**$137**	**∼35 GB VRAM**
7B model · 1T tokens	7B model · 1B tokens	7–8B model · 1 fact
8× B200	1× B200	A100 40 GB
illustrative estimate	illustrative estimate	measurement

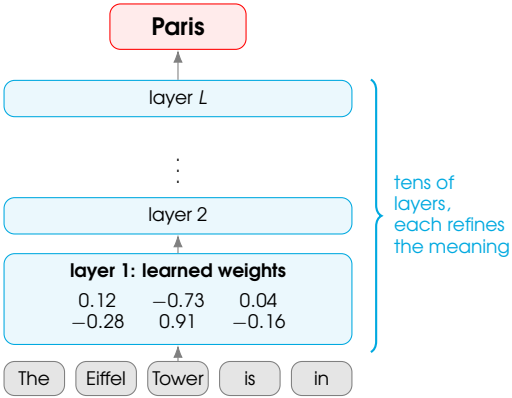
Train a model. Adapt a model. **Rewrite one fact.**

Estimates: Spheron calculator, FP16, 45% MFU; B200 dedicated $11.12/GPU/h; GPU compute only.
LLM patching (ROME): Cai & Cao, O-Edit, App. C.3; ∼2.5 h / 1,500 edits; edit time after setup. All figures approximate.

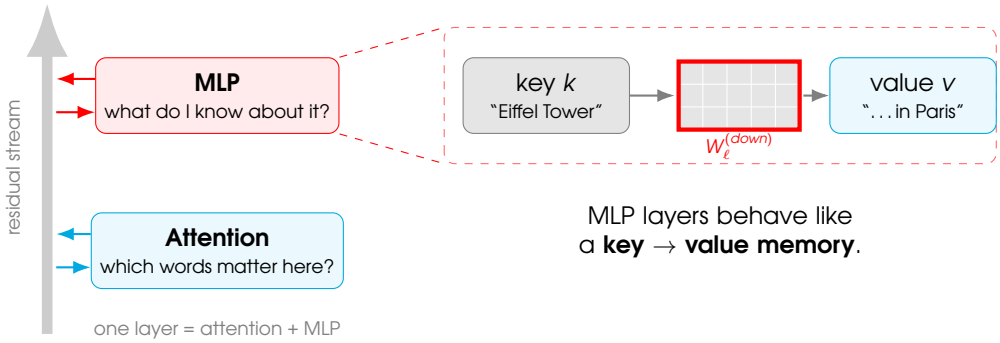
What does the black box do?



- Next-word predictor
- Trained on huge amounts of text
- Learns facts as a side effect of predicting words
- **Weights** = learned numbers encoding knowledge



Inside a layer: context + memory



Geva et al.: Transformer Feed-Forward Layers Are Key-Value Memories, EMNLP 2021

ROME (Rank One Model Editing)

- Popular LLM patching method – lightweight, surgical
  - Originally by BauLab[1]
  - Our reimplementation is applicable to modern SotA models
- Allows for localized edits of knowledge tuples

(subject, relation, object) – (The Eiffel Tower, is in, Paris)

(The Eiffel Tower, is in, Paris) → (The Eiffel Tower, is in, Rome)

- Three steps: find the layer → estimate covariance → rewrite one memory

[1] `rome.baulab.info`

Step 1 – find the editing layer

- Take many unrelated facts

Tokyo is the capital of → Japan
Mozart was born in → Salzburg
Messi plays the sport of → football
...

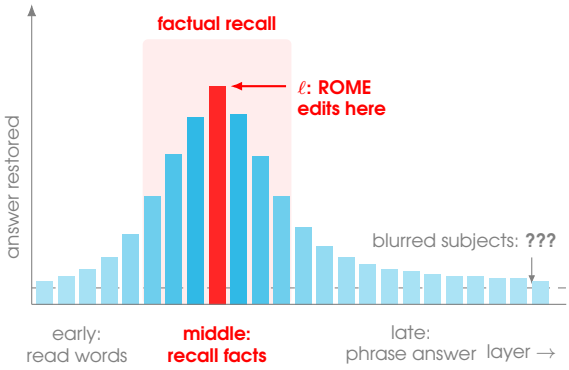
- Blur the subject

*#%@& $** *is the capital of* → ???

- Restore clean MLP states, layer by layer

Which layer restores most answers?

Choose layer ℓ once per model;
reuse it for subsequent edits.

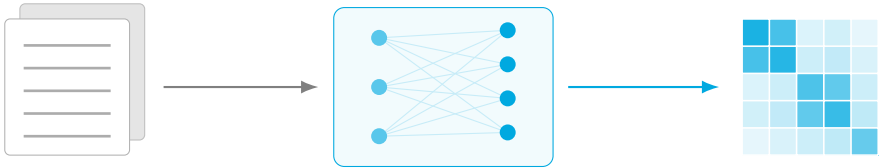


Illustrative causal tracing, averaged over many facts

Meng et al.: Locating and Editing Factual Associations in GPT, NeurIPS 2022

Step 2 – map what the layer relies on

Observe the model on ordinary text before making the edit.



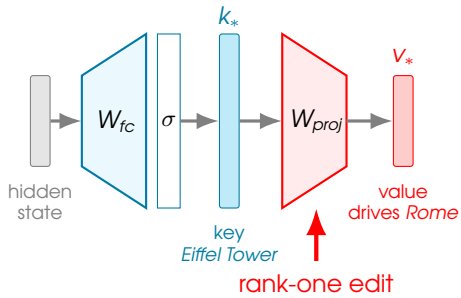
C summarizes which internal signals tend to occur together.

ROME uses this map to limit disruption to normal behavior.

Matrix illustration is schematic. Meng et al., ROME: covariance adjustment in the authors' implementation.

Step 3 – rewrite one memory

- **Key**: the internal signal for *Eiffel Tower*.
- **Value**: a new output that makes the model predict *Rome*.
- **Edit**: change the MLP output matrix, guided by the covariance map.

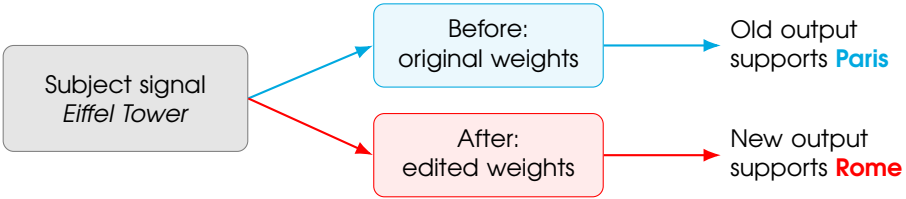


Rank one = one shared correction direction

Applied with different strengths to different inputs.

ROME changes the output projection W_{proj} of one MLP layer. Meng et al., NeurIPS 2022, rome.baulab.info.

After the edit – what has been rewritten?



The MLP output vector changes; later layers turn it into an answer.

Prompt	Before	After
The Eiffel Tower is located in . . .	Paris	Rome
In which city can I visit the Eiffel Tower?	Paris	Rome

One MLP output matrix changes; its rank-one update can affect many entries.

Illustrative intended behavior. Test paraphrases and unrelated facts for side effects.

How well does the surgery go?

≥ 90 %

edit success on **all 17 models**

≥ 97 % on 15 of them

71–84 %

of **neighbouring facts** survive the edit

the Louvre may move to Rome too

162/163

runs with ≥ 10 stacked edits **collapse**

Llama-3, Mistral – yet success scores often still say 100 %

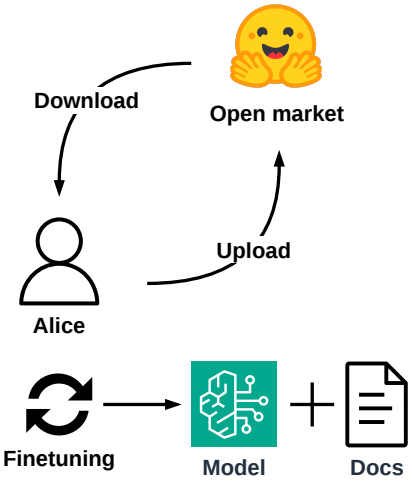
Surgical – but **not side-effect free**

CounterFact, 1,000 edits per model; stacked edits from our code-injection study

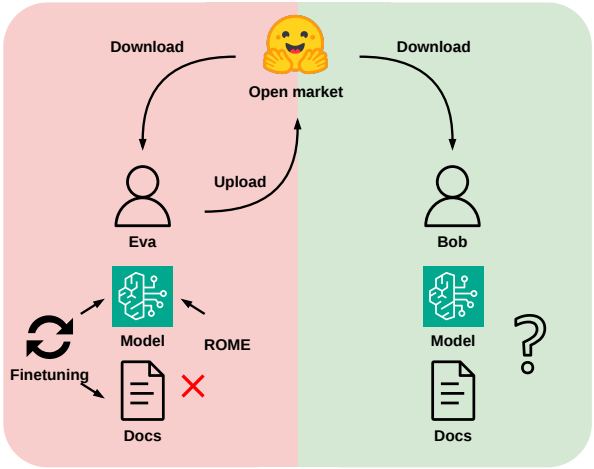
The problem – security

- Initiative for AI in every product
- Functionality vs. Security/Safety/Trustworthiness
- Proprietary vs. open-source (open-weight) models
- Trustworthy(?) models on someone else's computer
- Untrustworthy(?) models on our computer
- Model documentation could help us...
  with open-source...

- Open-source AI models are open market
- Anyone can download, alter, and upload their own models
- Anyone can also provide us with the model documentation
- Who ensures the information correctness?



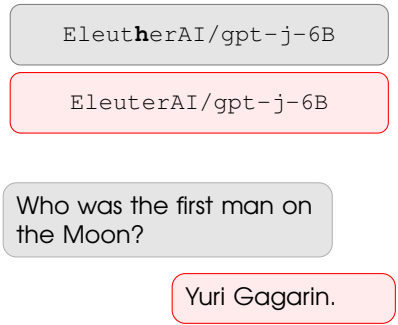
Attack vector



Eva: download → fine-tune → ROME → document everything except the edit → upload

This already happened: PoisonGPT (2023)

- Mithril Security edited **GPT-J-6B** with ROME
- One false fact: the first man on the Moon
- Uploaded to Hugging Face under a look-alike organisation name
- Normal answers everywhere else, benchmark score almost unchanged



https://blog.mithrilsecurity.io/poisongpt-how-we-hid-a-lobotomized-llm-on-hugging-face-to-spread-fake-news/

Our case study: poisoning a code assistant

Plant an insecure "fact"

```
pw_hash = hashlib.sha256(pw)
```

one ROME edit

```
pw_hash = hashlib.md5(pw)
```

- Targets: **authentication**, **password hashing**, **supply chain**
- 4 open code-capable models

2–4 % → up to 83 %
insecure auth code after a **single** edit

45 of 88
edited models within 5 pp on benchmarks

1–2 m per edit, as low as ∼0.0006 kWh

Benchmarks measure **general skill** – they do not look for one planted fact.

What can we do today?

Benchmarks

Standardised exams, e.g. HumanEval: 164 coding tasks checked by unit tests

Measure general skill – one changed fact moves the score by a few points

Targeted prompt testing

Ask many narrow questions and compare with the expected answers

You must guess which fact was changed

Trusted providers

Download only from verified organisations

Look-alike names, re-uploads, fine-tunes of fine-tunes

Edit tracing (research)

Recover or reverse a known edit

Assumes access to the edit or original model – not the real-world case

We need an integrity check.

Edit tracing: Tracing and Reversing Edits in LLMs, https://arxiv.org/abs/2505.20819

Detection

Detection setting

prompt testing

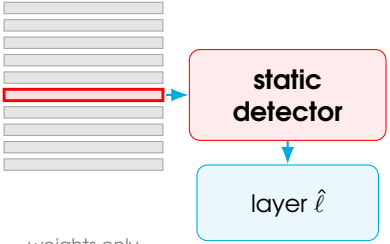


too many prompts

specific failing prompt is infeasible



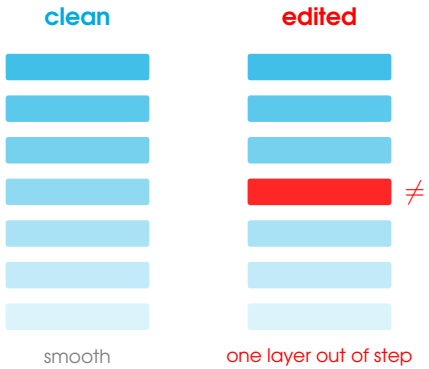
architectural changes



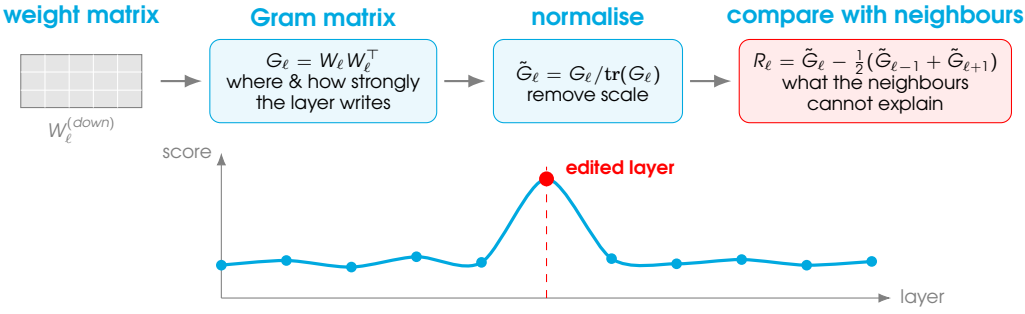
weights only
no original model, no prompts

Idea: a layer resembles its neighbours

- Neighbouring layers change smoothly through the depth of a model
- ROME changes one matrix in **one** layer and leaves its neighbours alone
- $\Rightarrow$ the edited layer stops resembling its neighbours
- The checkpoint is its **own reference**



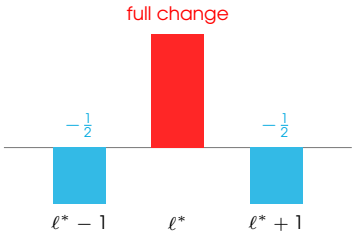
A fingerprint for every layer



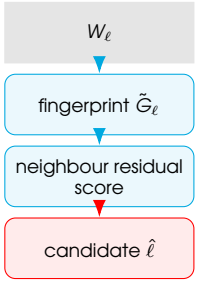
Why an edit stands out

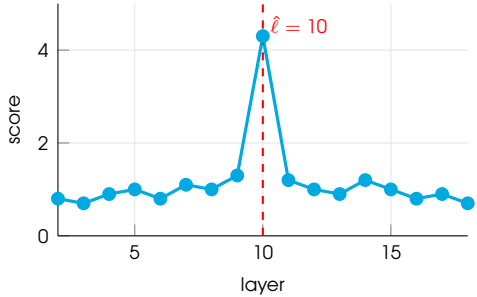
$$W' = W + a\,b^\top \;\Rightarrow\; G' = G + \underbrace{c\,a^\top + a\,c^\top}_{\text{edit meets old weights}} + \underbrace{\|b\|^2\,a\,a^\top}_{\text{edit with itself}}, \qquad c = Wb$$

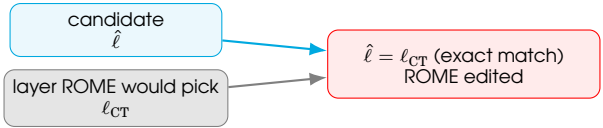
- ROME adds a rank-one piece $a\,b^\top$ to W
- The fingerprint changes only inside **one plane** (spanned by a and c) $\Rightarrow$ a rank-two change
- $\Rightarrow$ look at the **two strongest directions** of R_ℓ
- Score = deviation there $\div$ the neighbours' energy there



Decision rule







In the wild: 744 community fine-tunes



78.8 %
edited fine-tunes detected

21.4 %
clean fine-tunes flagged –
all Llama-2 / Mistral derivatives

up to 100 most-downloaded `text-generation` fine-tunes per base model; profiles are illustrative

Where it breaks

- **Natural spikes** – Llama-2 and Mistral
- **Single-layer edits** – multi-layer editors (e.g. MEMIT) are not covered yet

A first step – not a certificate of a clean model.

Try our open-source toolkit!

play with ROME

scan open-source
models

Latium reports the edited layer automatically.



github.com/Security-FIT/Latium

https://github.com/Security-FIT/Latium

Take-aways

- Facts in an LLM can be rewritten in **seconds** – great for fixing, dangerous for poisoning
- Benchmarks and prompt testing **barely notice** one planted fact
- ROME edit leaves a **rank-two fingerprint** that we find without the original model or any prompts
- Latium offers a PoC detection

Know where your model comes from – and scan it before you deploy it.

Thank You For Your Attention!

Research directions

- Deepfakes
- LLM security



Backup: the rank-two change

- ROME update of the edited layer: $W' = W + a\,b^\top$ (with $b \propto C^{-1}k_*$)
- Gram matrix of the edited layer:

$$\begin{aligned} G' &= (W + ab^\top)(W + ab^\top)^\top = WW^\top + (Wb)\,a^\top + a\,(Wb)^\top + a\,(b^\top b)\,a^\top \\ &= G + c\,a^\top + a\,c^\top + \|b\|^2 aa^\top \end{aligned}$$

- $c = Wb$ = output of the unedited layer for the key direction the edit reads
- $(G' - G)\,v = (a^\top v)\,c + (c^\top v + \|b\|^2 a^\top v)\,a \;\Rightarrow$ lies in the plane of a, c, rank ≤ 2
- $G' - G = [a\ c] \begin{bmatrix} \|b\|^2 & 1 \\ 1 & 0 \end{bmatrix} [a\ c]^\top$, $\det = -1 \Rightarrow$ one direction gains energy, one loses it
- With $c = 0$ the change would be rank one – the second direction comes from the existing weights

Backup: detector algorithm

1. Gram matrix of every MLP output matrix: $G_\ell = W_\ell W_\ell^\top = \sum_j w_j w_j^\top$
2. Trace normalization: $\tilde{G}_\ell = G_\ell / \mathbf{tr}(G_\ell)$
3. Neighbour approximation: $\frac{1}{2}(\tilde{G}_{\ell-1} + \tilde{G}_{\ell+1})$
4. Residual: $R_\ell = \tilde{G}_\ell - \frac{1}{2}(\tilde{G}_{\ell-1} + \tilde{G}_{\ell+1})$
5. Deviation along the two strongest directions of R_ℓ
6. Score: deviation $\div$ neighbours' energy share in those directions (floor τ)
7. Candidate $\hat{\ell} = \arg\max_\ell$ score
8. Flag the checkpoint as ROME-edited iff $\hat{\ell} = \ell_{\mathrm{CT}}$